CENTRALE NANTES

Projet SMP Mai 2026

itii PAYS DE LA LOIRE

Les Aventuriers du Rail

Implémentation C++ d'un jeu de société

Baptiste - Yann - Lisa - Olwen

SOMMAIRE

01
Le projet
Contexte, équipe, objectifs

02
Les règles du jeu
D'après le sujet du TP

03
Architecture - Diagramme UML
Classes, relations, organisation

04
Les outils utilisés
Git, CMake, Doxygen

05
L'interface Qt (bonus)
Affichage graphique du plateau

06
La boucle de jeu
Rôle du main, actions possibles à chaque tour, actions du main

07
Conclusion
Difficultés, apprentissages, bilan

01 - Le projet

Objectifs :

Recréer le jeu de société **Les Aventuriers du Rail** en C++, en utilisant la collaboration avec Github, la compilation avec CMake, et la documentation avec Doxygen.

Les objectifs principaux étaient :

- Créer un jeu fonctionnel
- Maîtriser les outils du développement logiciel (C++, CMake, Doxygen)
- Collaboration d'un projet C++ en équipe avec Git.

C++
Language de programmation

Doxygen
Documentation

CMake
Compilation

Qt6
Affichage

GitHub
Gestion du code en équipe

L'équipe :

Baptiste De Paul

Yann Buttet

Lisa Le Goff

Olwen Pichard

02 - Les règles du jeu

Objectifs

Relier des villes via des tickets destination en posant des wagons sur des routes.

Cartes train

72 cartes (6 couleurs × 10 + 12 locomotives joker) mélangées et distribuées.

Poser des wagons

Défausser N cartes train de la bonne couleur pour revendiquer une route de longueur N.

Tickets

Chaque joueur doit relier 2 villes précises. Le ticket validé rapporte 1 point.

Routes doubles

Certaines routes ont deux voies parallèles de couleurs différentes.

Fin de partie

Le premier à valider 6 tickets remporte la partie ou celui qui a validé le plus de ticket lorsqu’un des joueurs n’a plus de wagons.

03 - Architecture - Diagramme UML

Modèle du jeu

Plateau

Ville

Route

Joueurs

Joueur

Cartes & Tickets

CarteTrain

Ticket

Couleurs (enums)

CouleurCarte

CouleurRoute

CouleurWagon

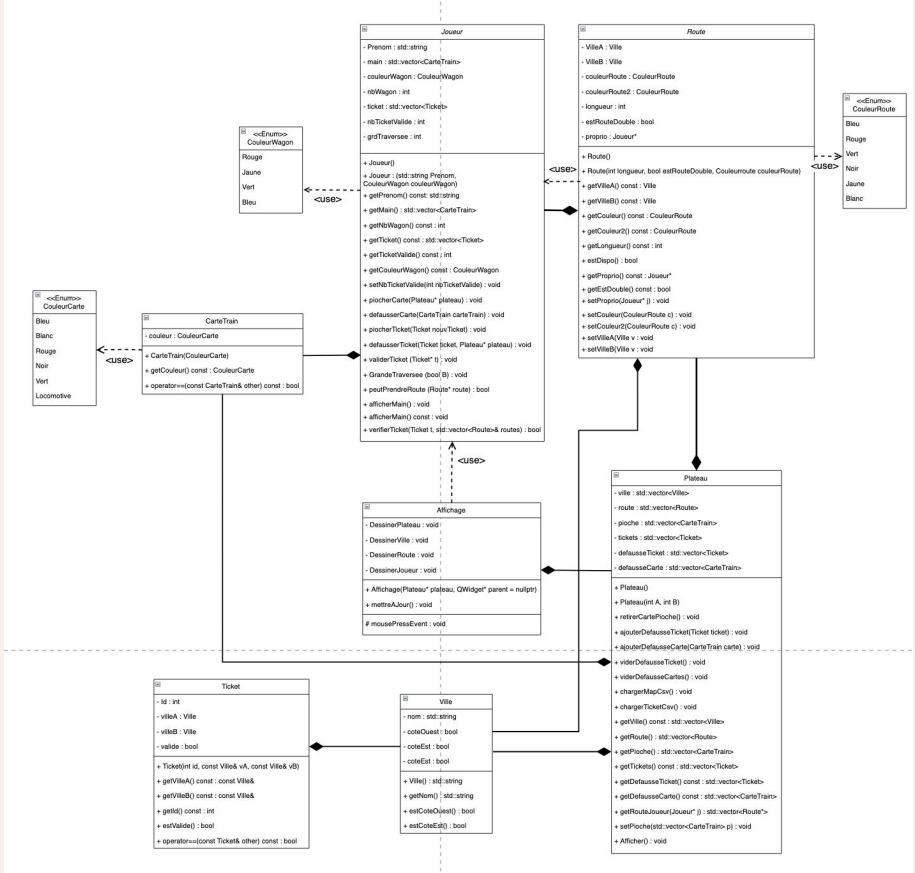
Interface Qt

Affichage

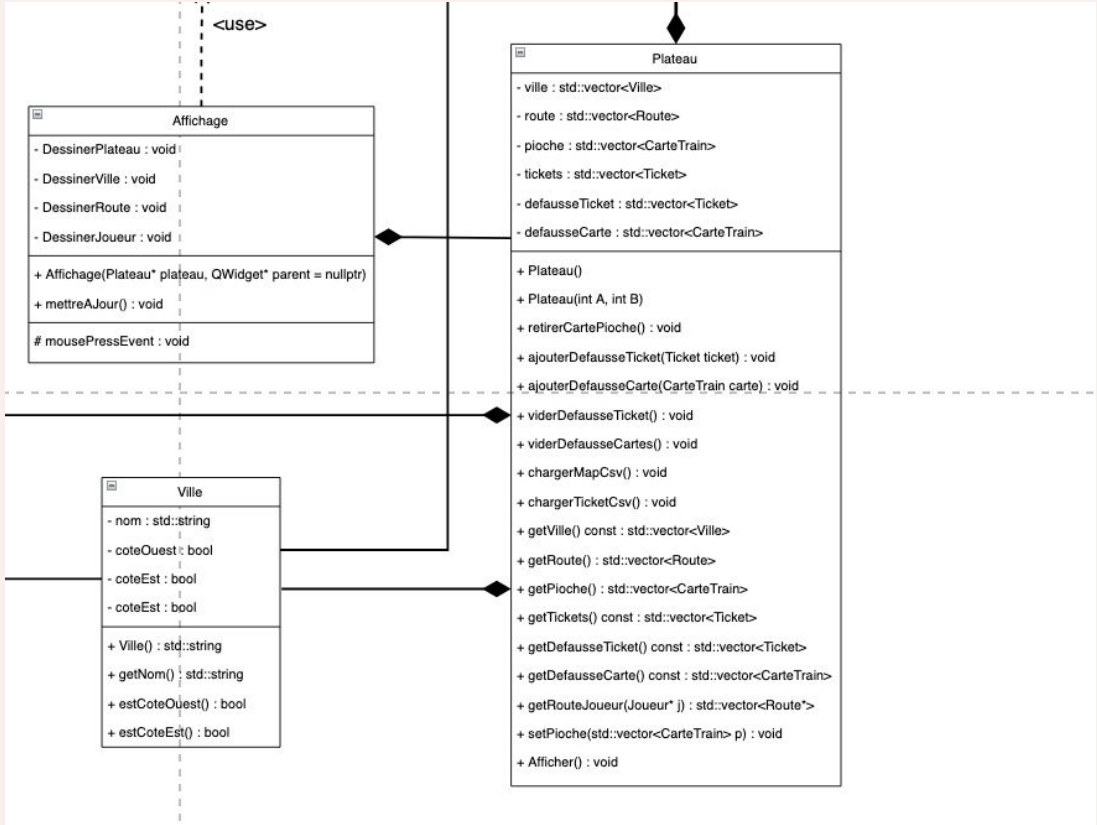
- *Plateau contient ⟶ Routes, Villes, CarteTrain, Tickets*
- *Joueur interagit avec ⟶ Plateau, Route, CarteTrain, Ticket*
- *Affichage lit ⟶ Plateau, Joueur, Route*

03 - Architecture - Diagramme UML

- 10 classes
- 2 types de relations



03 - Architecture - Diagramme UML

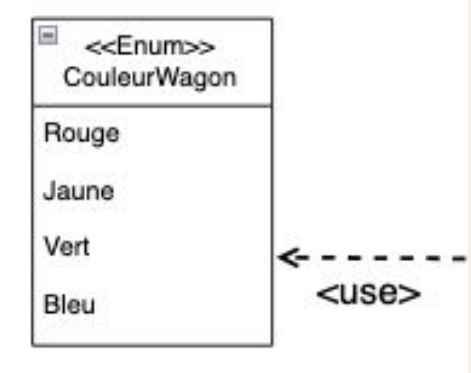


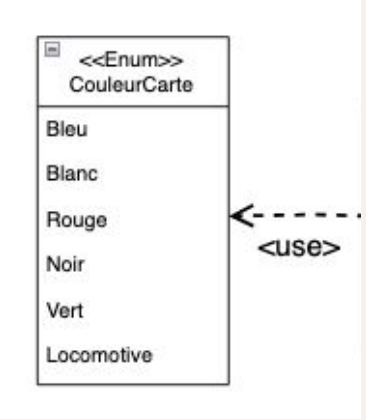
Plateau

Place centrale

stocke:
Villes, Routes, Pioches et Tickets

03 - Architecture - Diagramme UML



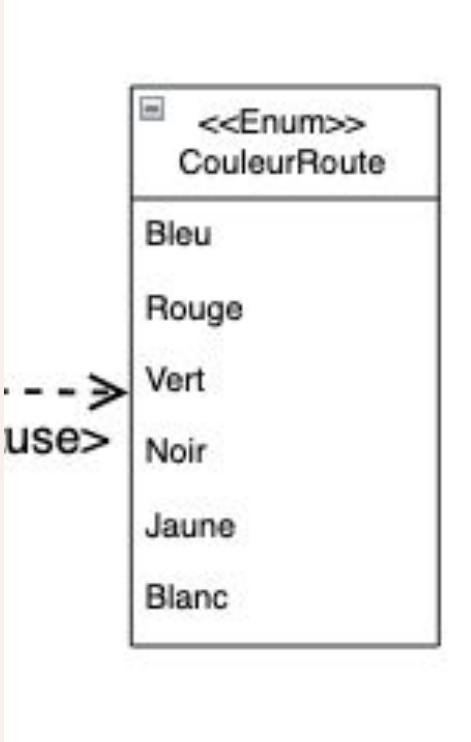


Classes enum

Pas objet mais types

Listes de valeurs

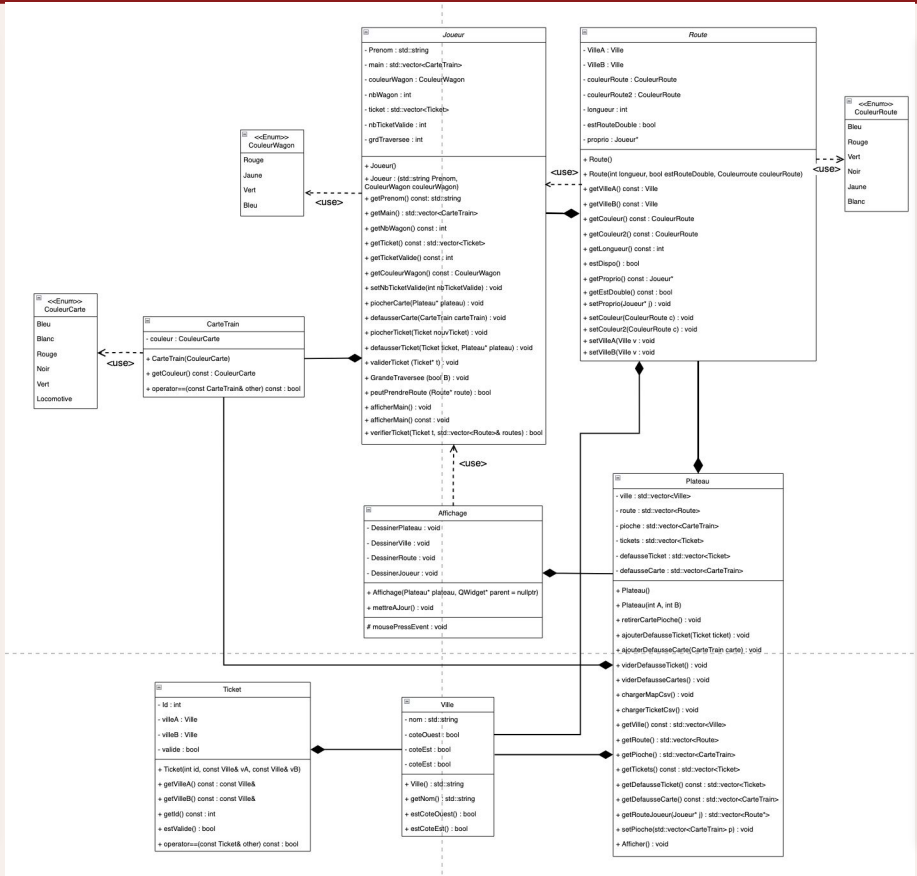
Évite d'écrire 1 pour bleu, 2 pour rouge…



03 - Architecture - Diagramme UML

Agrégations

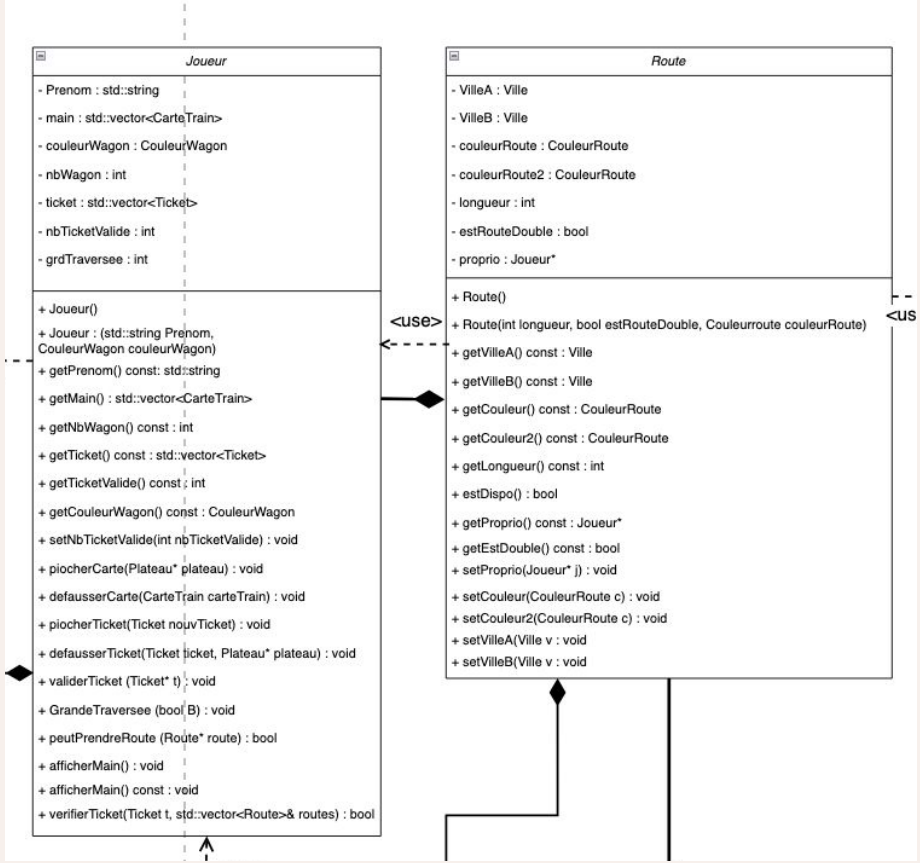
- Trait plein avec diamant.
- Une classe en contient une autre dans ses attributs.
- Exemple: Plateau
- Si le plateau disparaît, tout part avec lui.



Dépendances

- Trait pointillé avec flèche.
- Pas d'appartenance mais utilisation ponctuelle.
- En paramètre d'une méthode ou en variable.
- Exemple: Affichage
- Même chose pour les enums.

03 - Architecture - Diagramme UML



Lien Route - Joueur

- Route a un attribut **proprio** qui est un pointeur vers le joueur qui la possède.
- Joueur utilise Route pour vérifier s'il peut la prendre.
- **Problème:** Inclusion circulaire
  => erreur de compilation lors des test
- **Solution:** Forward déclaration (pour dire au compilateur que la classe existe déjà et qu'elle sera définie plus tard, sans inclure header).
- Dans Route: seulement la classe Joueur;

04 - Les outils utilisés

Git & GitHub

- Travail en branches (une par membre de l'équipe et une branche plus générale "main")
- Pull requests et merge pour intégrer le travail
- Résolution de conflits à la main
- Historique complet des modifications
- Commit à chaque nouvelle modification pour le suivi du projet

CMake

- Compilation sur toutes les plateformes (Linux, macOS, Windows)
- Gestionnaire de dépendances (pour l'intégration Qt6 et Doxygen)
- Génération du build
- Pour des tests unitaires (non utilisé)

Doxygen

- Documentation générée depuis les commentaires
- Groupes de modules (Modèle, Joueurs, Cartes…)
- Page d'accueil depuis le README.md

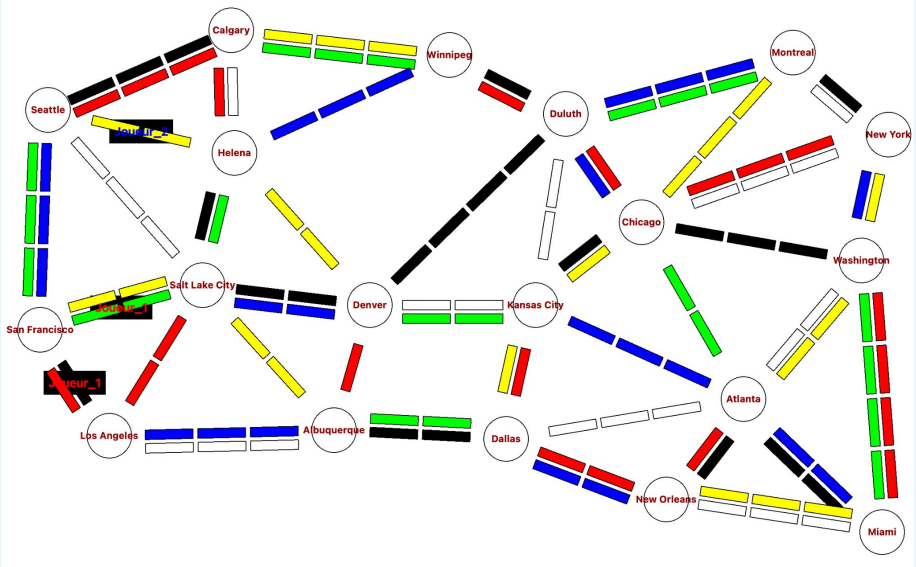
05 - L'interface Qt

Un ajout bonus pour plus de visualisation

La version console était fonctionnelle, mais nous avons voulu ajouter une interface graphique avec Qt pour mieux visualiser notre programmation.

- **QGraphicsScene** Dessin 2D du plateau, villes et routes
- **Routes colorées** Chaque couleur correspond aux cartes requises
- **Routes doubles** Deux voies parallèles visibles sur la carte
- **Propriétaire** Le prénom du joueur s'affiche sur sa route
- **Mise à jour live** L'affichage se rafraîchit à chaque action

Plateau de jeu



Capture d’écran du plateau de jeu

06 - La boucle de jeu (main.cpp)

Rôle du main

Le “main” est le point d’entrée du programme. Il initialise le jeu, lance la partie, dialogue avec le joueur

Les 3 actions possibles par tour :

- Piocher une carte train
- Poser des wagons pour acquérir une route
- Échanger 2 tickets contre 2 nouveaux

Actions du main

1

Initialisation

Joueur, pioche de cartes et tickets chargés depuis le CSV

2

Boucle de jeu

Chaque joueur choisit l’une des 3 actions, l’un après l’autre

3

Fin de partie

Quand un joueur valide 6 tickets ou ne possède plus aucun wagon

06 - Précision sur le jeu

Mélange aléatoire

Algorithme de Fisher-Yates appliqué à la pioche de cartes et de tickets via rand() + srand(time).

Validation des tickets

Après chaque pose de wagons, le programme vérifie si un ticket est complété et le valide si c'est la cas.

Test global

Les tests ont été effectué au fur et à mesure de la programmation, sans test unitaire.

Qt réactive

app.processEvents() appelé après chaque action pour garder la fenêtre graphique à jour en temps réel.

07 - Conclusion

Ce qui nous a posé problème :

- Gérer les inclusions circulaires (Joueur ↔ Route ↔ Plateau)
- Synchroniser Qt et la logique console (processEvents)
- Résoudre les conflits Git à plusieurs
- Calculer les rotations des segments de routes, car un rectangle dans Qt est toujours horizontal donc besoin d'être pivoté pour relier deux villes.
- Comprendre la quantité de travail et répartir les tâches pour avancer efficacement
- Vérifier si une route relie bien deux villes (algorithme Breadth-First Search (BFS) pour parcourir le graphe en largeur)

Ce que nous avons appris :

- Maîtriser CMake
- Documenter avec l'outil Doxygen
- Gérer Git en équipe, branches, merge, résolution de conflits
- Qt6 / QGraphicsScene, Affichage 2D, transformations (aide IA)

Merci !

Des questions ?

Baptiste - Yann - Lisa - Olwen